Artificial Neural Network

Yaya Wihardi, S.Kom., M.Kom.

Email: yayawihardi@upi.edu

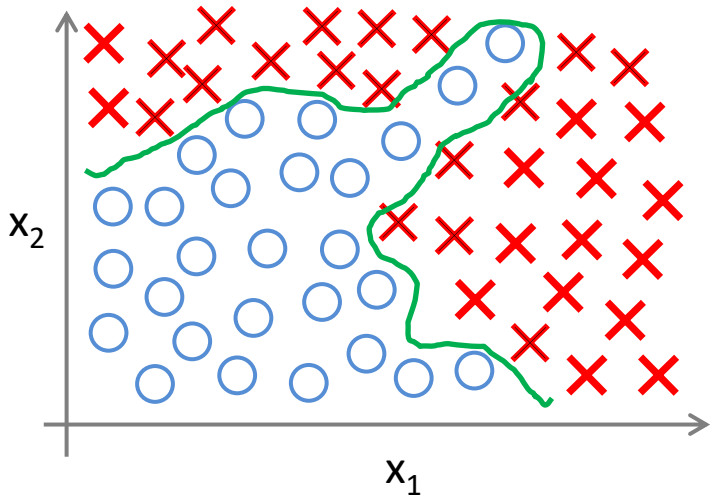
Department of Computer Science Education

Universitas Pendidikan Indonesia

Outline

- Non-Linearity
- Model Representation 1
- Model Representation 2
- Multi Class Classification
- ANN: Training Model

Non-Linearity



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

x_1 = size
x_2 = # bedrooms
x_3 = # floors
x_4 = age
$\dots$
x_{100}

Kombinasi fitur-fitur yang ada (feature map):

$x_1, x_2, x_3, \dots, x_{100}$ → 100 fitur

$x_1^2, x_1x_2, x_1x_3, \dots, x_2^2, x_2x_3, x_2x_4, \dots, x_{100}^2$ → 5000-an fitur

bagaimana dengan pangkat yang lebih besar?

Non-Linearity di Dunia Nyata

Yang Kita Lihat:

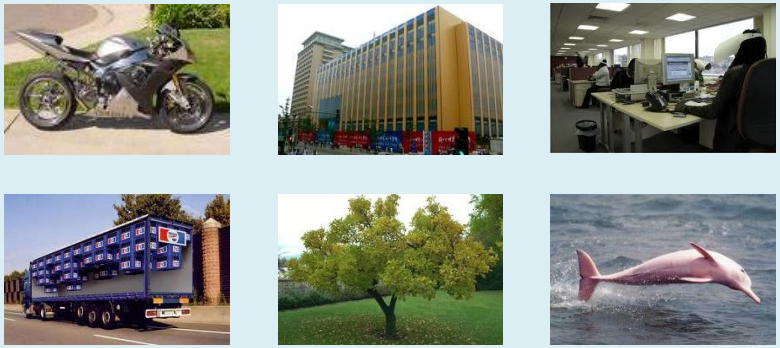
Yang Komputer Lihat:

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50

Contoh Kasus Computer Vision: Car Detection

Data Training

Cars

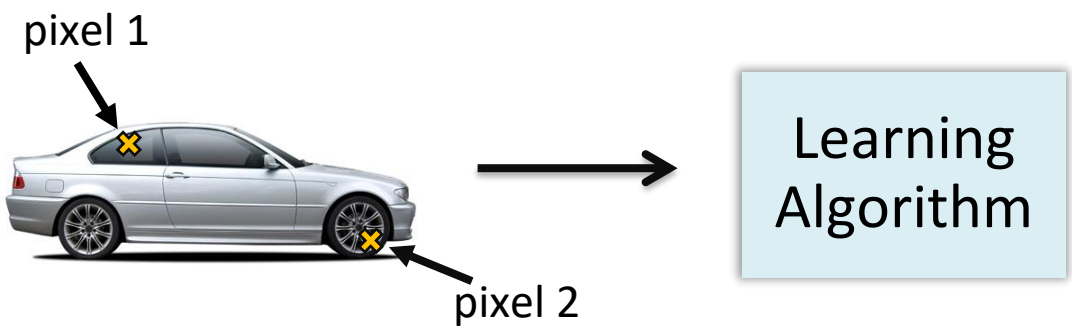


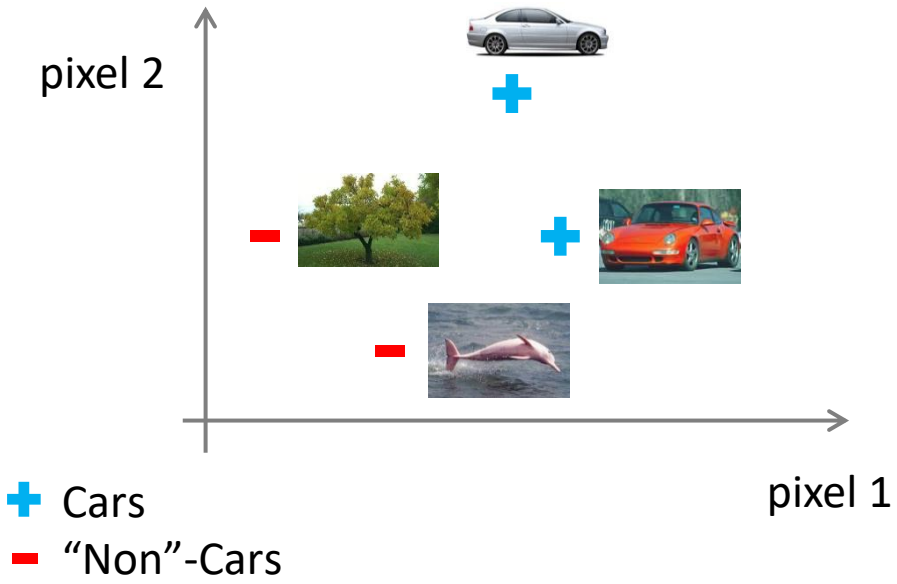
Not a car

Testing:

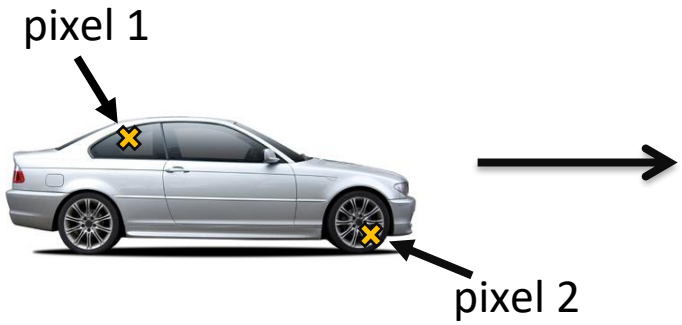
What is this?

Contoh Kasus Computer Vision: Car Detection

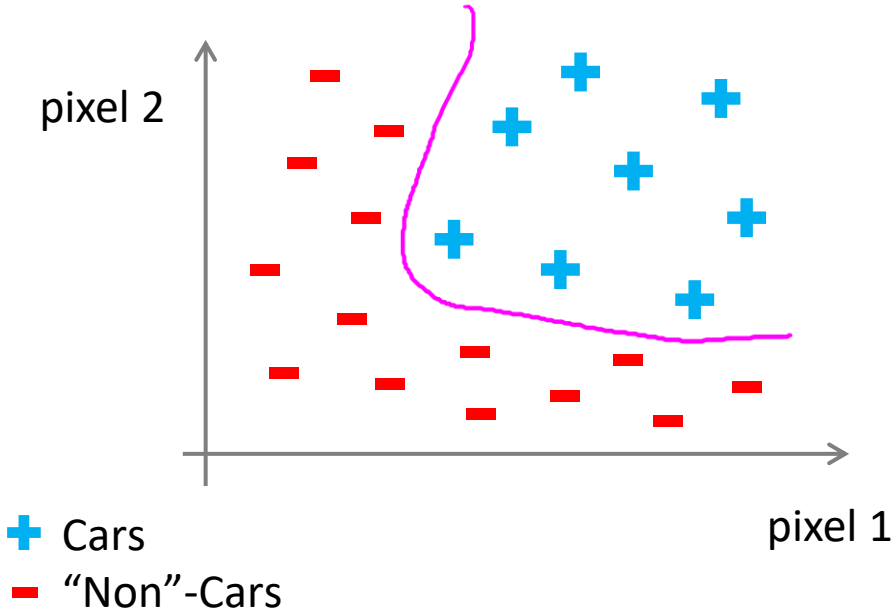




Contoh Kasus Computer Vision: Car Detection



Learning Algorithm



50 x 50 pixel → 2500 pixels

$n = 2500$ (7500 utk RGB)

$$x = \begin{bmatrix} \text{pixel 1 intensity} \\ \text{pixel 2 intensity} \\ \vdots \\ \text{pixel 2500 intensity} \end{bmatrix}$$

Quadratic features ($x_i \times x_j$): ≈3 million features

Artificial Neural Network (ANN)

- Model yang meniru cara kerja sistem otak manusia
- Banyak digunakan di tahun 80an s.d. 90an, namun sempat redup popularitasnya di akhir tahun 90an
- Saat ini menjadi state-of-the-art dalam berbagai masalah dan domain machine learning

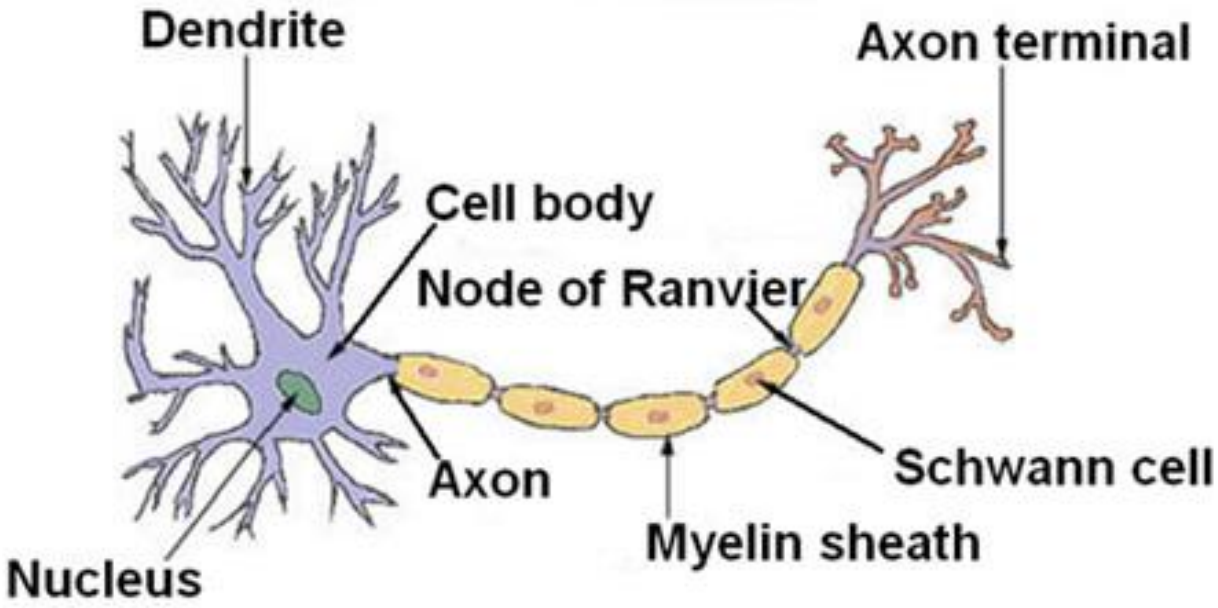
Sejarah Perkembangan

- 1943: Perceptron model (McCulloch & Pitts) → Linear Classifier
- 1958: Implementation as Mark I Perceptron (Rosenblatt) → handwritten letter recognition
- 1969: Perceptrons cannot learn XOR (Minsky & Papert)
- 1985: Representation learning (Rumelhart, Hinton, & Williams)
- 1989: Convolutional neural networks (Lecun)
- 1995: Long short-term memory (Hochreiter & Schmidhuber)
- 1998: Convolutional neural networks for MNIST (Lecun)
- 2012: ImageNet breakthrough (Krizhevsky, Sutskever, & Hinton)
- 2017: Transformer architecture (Vaswani et al.)
- 2018: Turing award (Bengio, Hinton, & Lecun)

What's New?

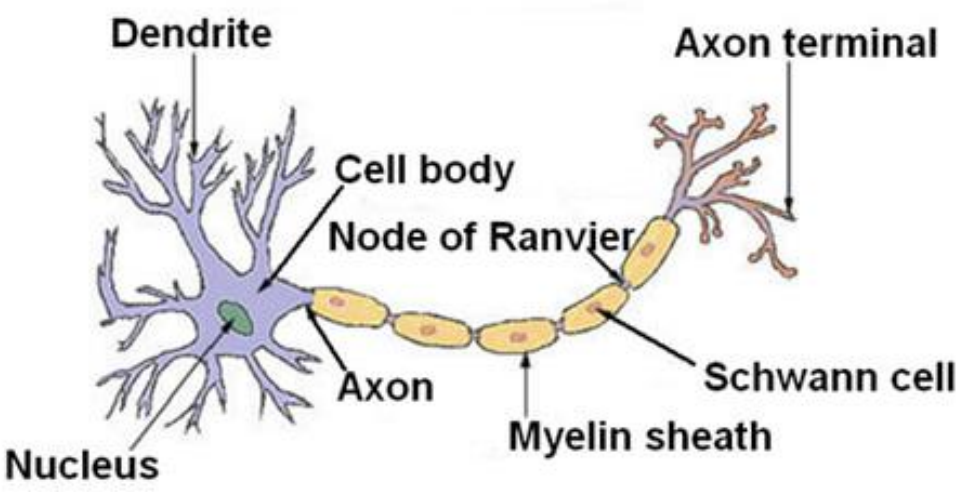
- More compute: GPUs, TPUs
- More data: ImageNet, Ms. COCO, Wikipedia/Web, etc.
- Better optimization algorithms: Mini-batch SGD, acceleration, etc.
- Accumulation of "folk knowledge": Parameter initialization (Transfer Learning), etc; Encoded in open source software packages
- Modern perspective: "Differentiable programming"
- Investasi besar-besaran baik di Industri, maupun di pemerintahan
- Transformer

Inspirasi Representasi Model: Human Brain

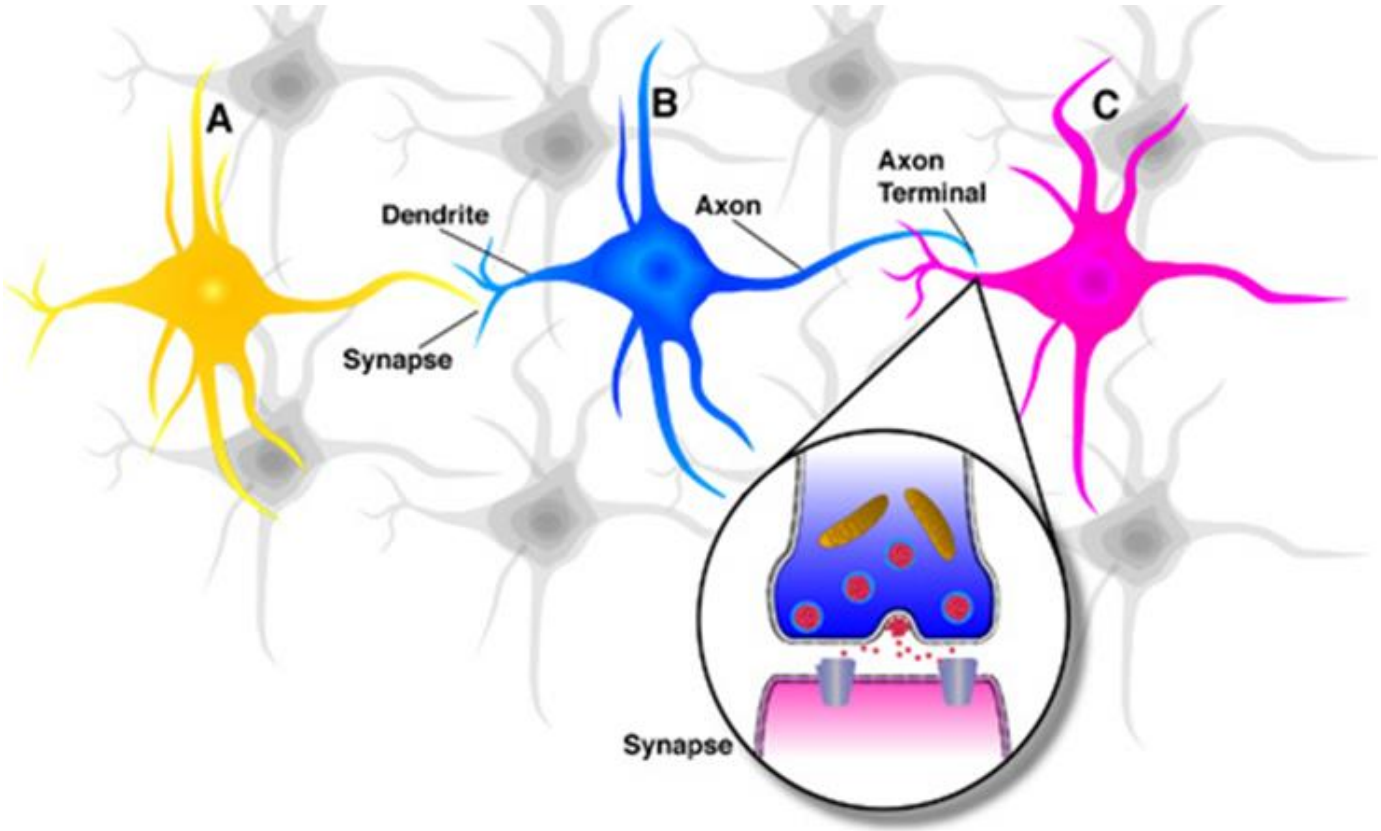


Human Brain

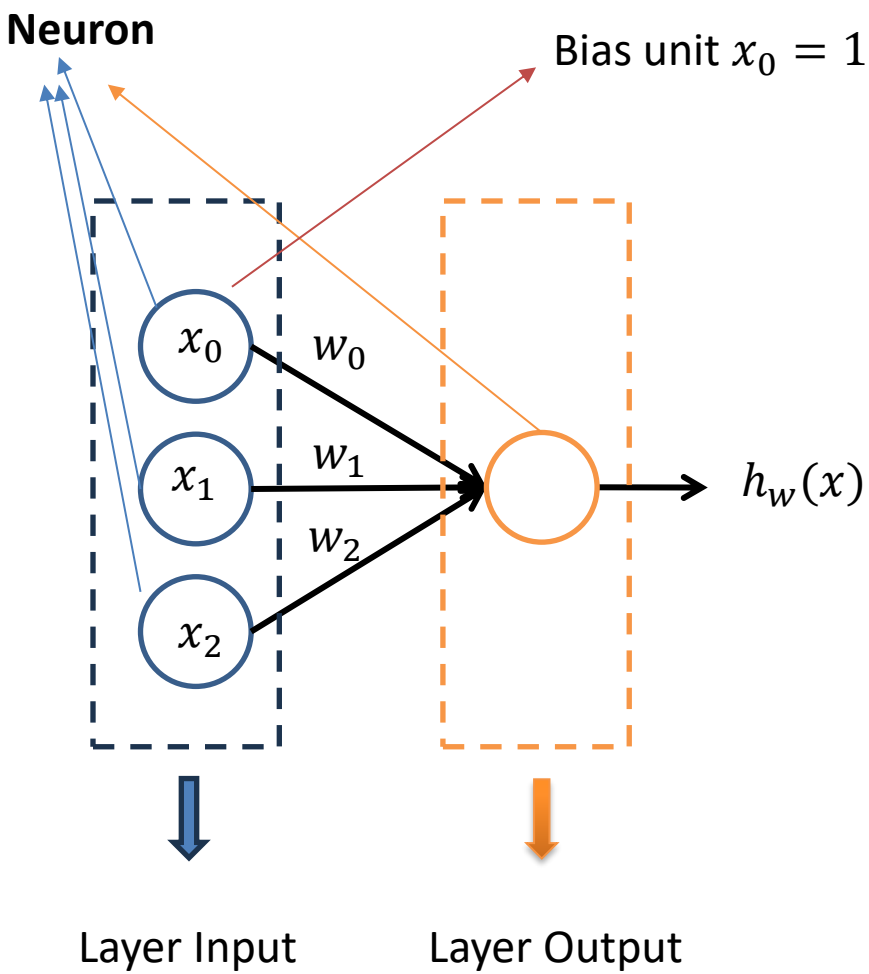
- Bertugas untuk memproses informasi
- Seperti prosesor sederhana
- Masing-masing cell tersebut berinteraksi mendukung kinerja otak
- Setiap sel (neuron) memiliki satu nukleus (soma), bertugas memproses informasi, informasi diterima oleh dendrit, dan disebarkan melalui akson
- Pertemuan informasi antar syaraf berada di sinapsis



Human Neuron



Neuron Model: Regresi Linear



Data Input

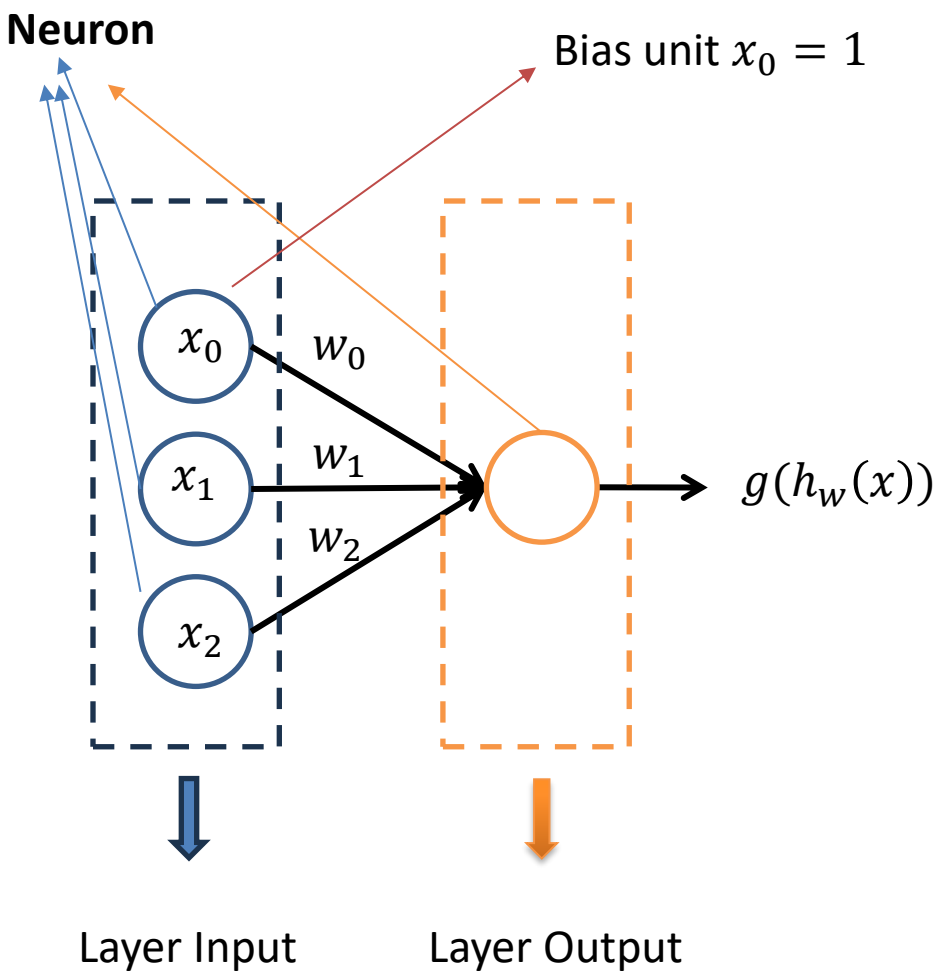
$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

Parameter/ Bobot

$$w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

$$h_w(x) = w_0 x_0 + w_1 x_1 + w_2 x_2$$

Neuron Model: Logistic Regression



Data Input

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

Parameter/ Bobot

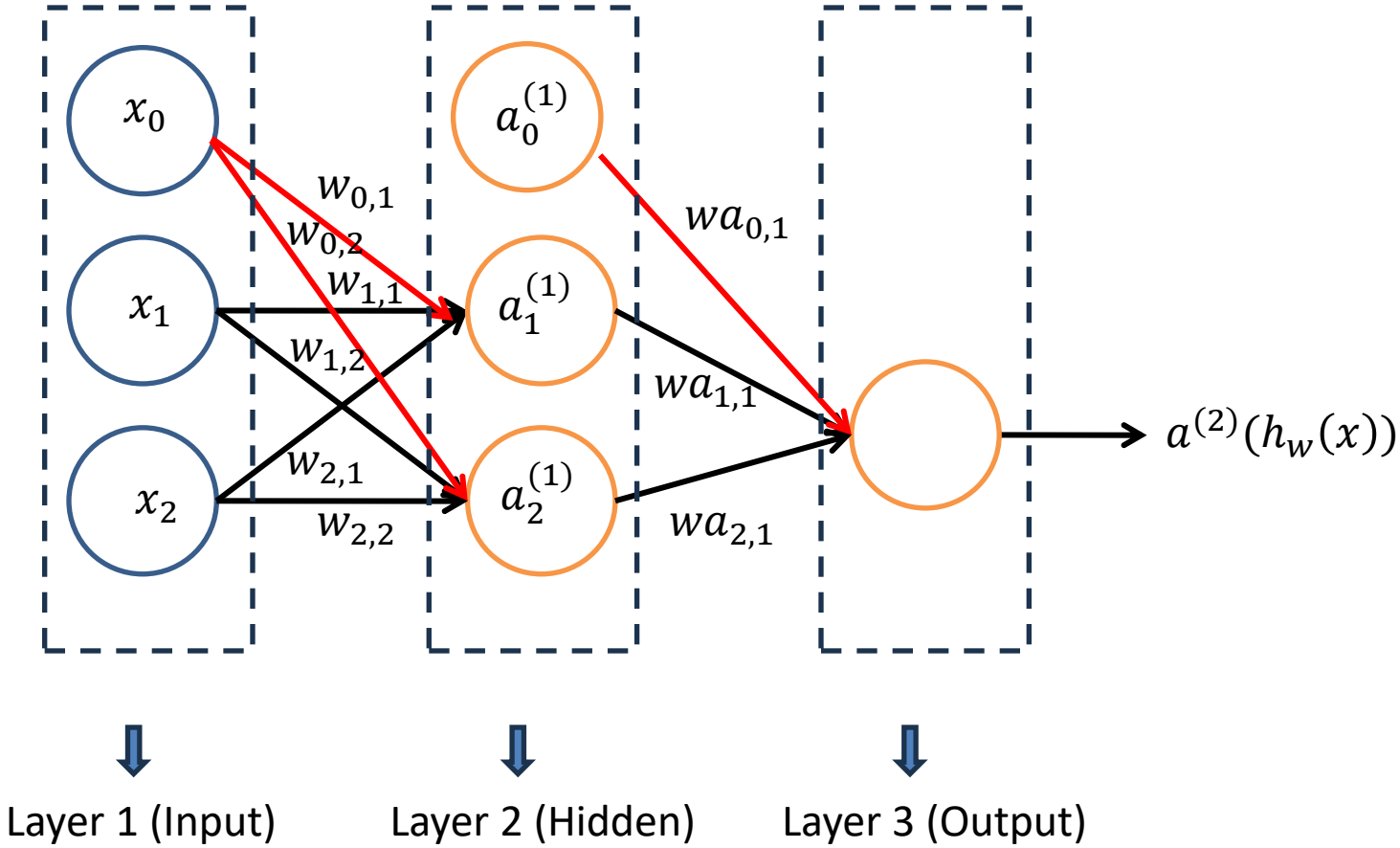
$$w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

$$\boxed{g(z) = \frac{1}{1 + e^{-z}}}$$

$$h_w(x) = w_0 x_0 + w_1 x_1 + w_2 x_2$$

Activation Function

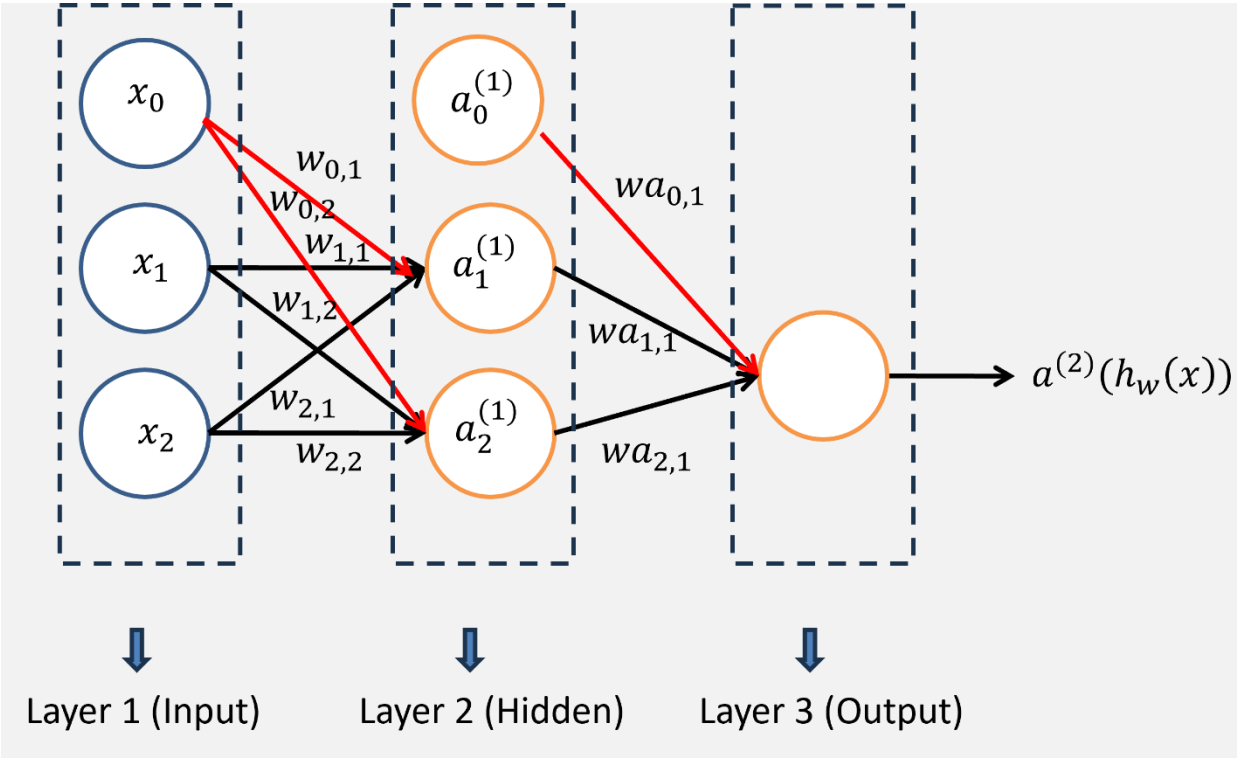
Neural Network Model Representation



NN: Model Representation

act_1 : *Activation Function* 1

act_2 : *Activation Function* 2



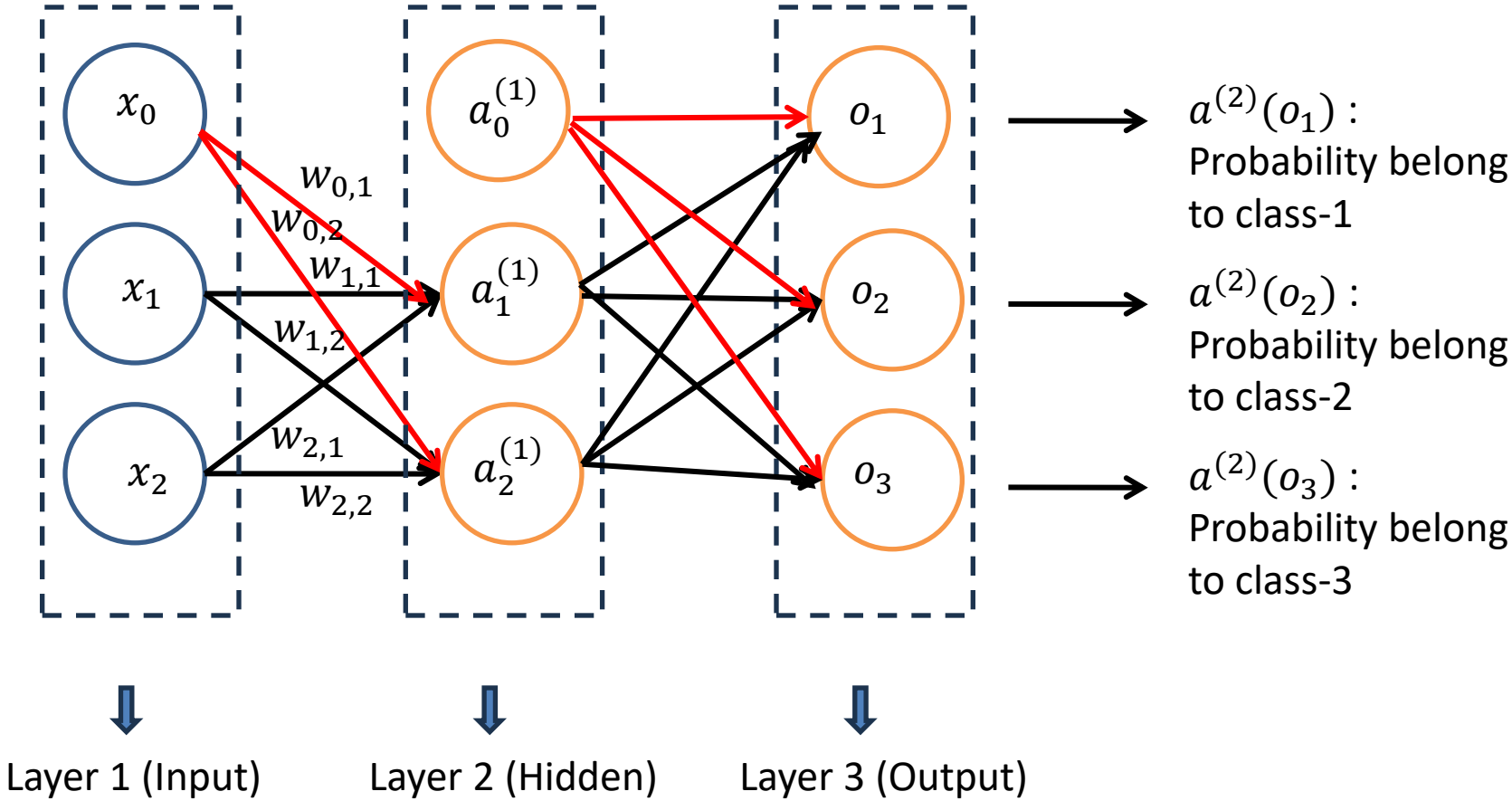
$$a_0^{(1)} = 1$$

$$a_1^{(1)} = act_1(w_{0,1}x_0 + w_{1,1}x_1 + w_{2,1}x_2)$$

$$a_2^{(1)} = act_1(w_{0,2}x_0 + w_{1,2}x_1 + w_{2,2}x_2)$$

$$a^{(2)} = act_2(wa_{0,1}a_0^{(1)} + wa_{1,1}a_1^{(1)} + wa_{2,1}a_2^{(1)})$$

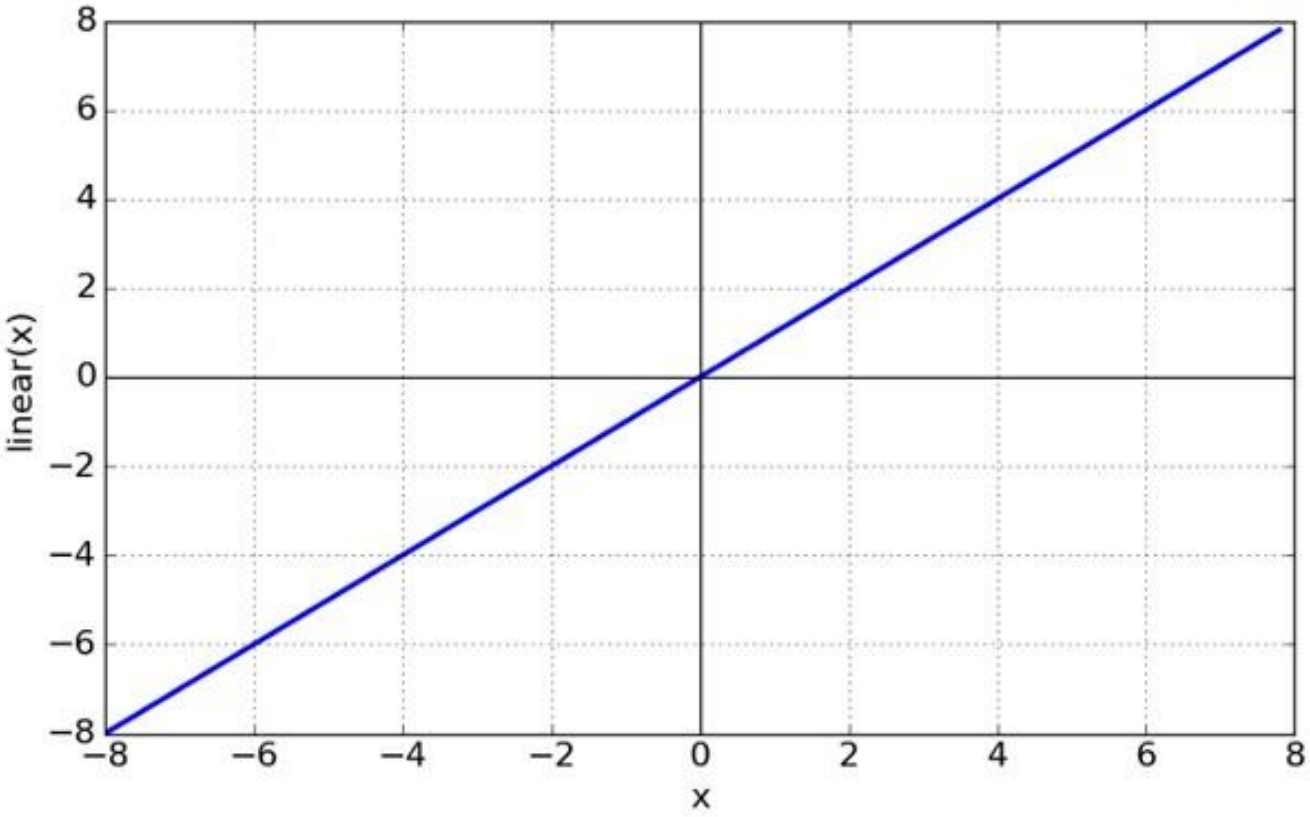
Neural Network: Multiclass Classification



Fungsi Aktivasi:

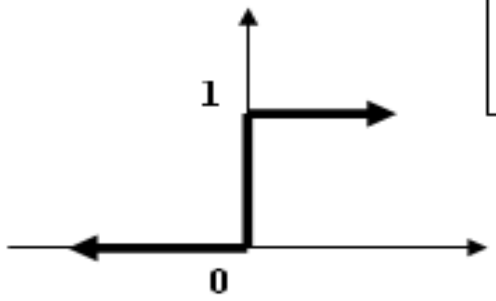
- Linear
- Non Linear
  - Hard Limit, Thresholding
  - Bipolar
  - Sigmoid: output range dari 0 s.d. 1 cocok utk estimasi probabilitas, klasifikasi 2 kelas
  - Softmax: output range 0 s.d. 1, cocok utk multiclass
  - Tanh: output range dari -1 s.d. 1
  - ReLU: output range 0 s.d. tak hingga, kekurangannya semua nilai nigatif akan menjadi 0, banyak digunakan pada Deep Learning
  - Leaky ReLU: improvement dari ReLU, range –tak hingga s.d. tak hingga

Fungsi Aktivasi Linear: f(x) = x



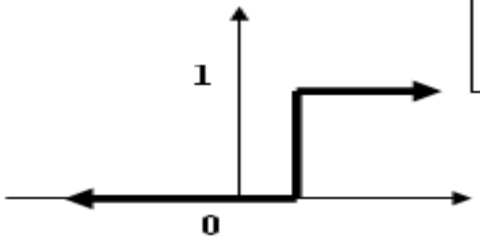
Fungsi Aktivasi Non-Linear

- Fungsi undak biner (hard limit)



$Y = 0$, jika $x <= 0$
$Y = 1$, jika $x > 0$

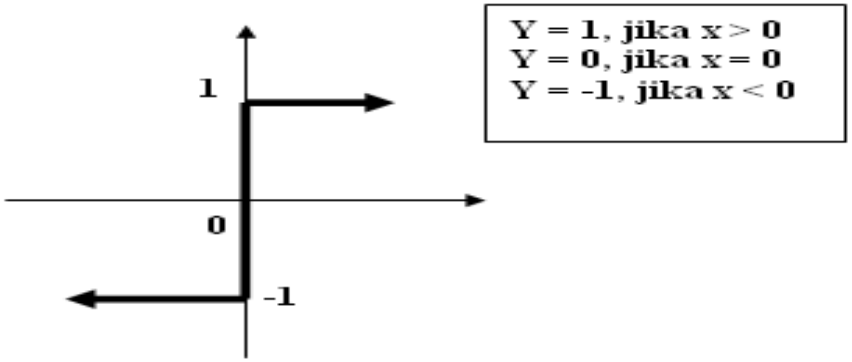
- Fungsi undak biner (threshold)



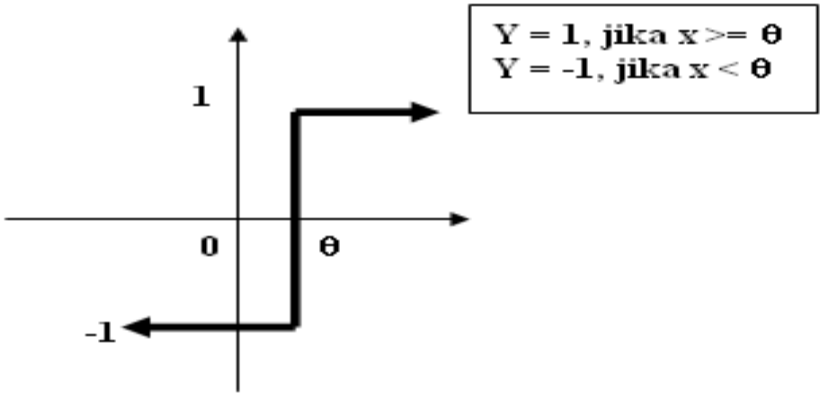
$Y = 0$, jika $x < \theta$
$Y = 1$, jika $x >= \theta$

Fungsi Aktivasi Non Linear

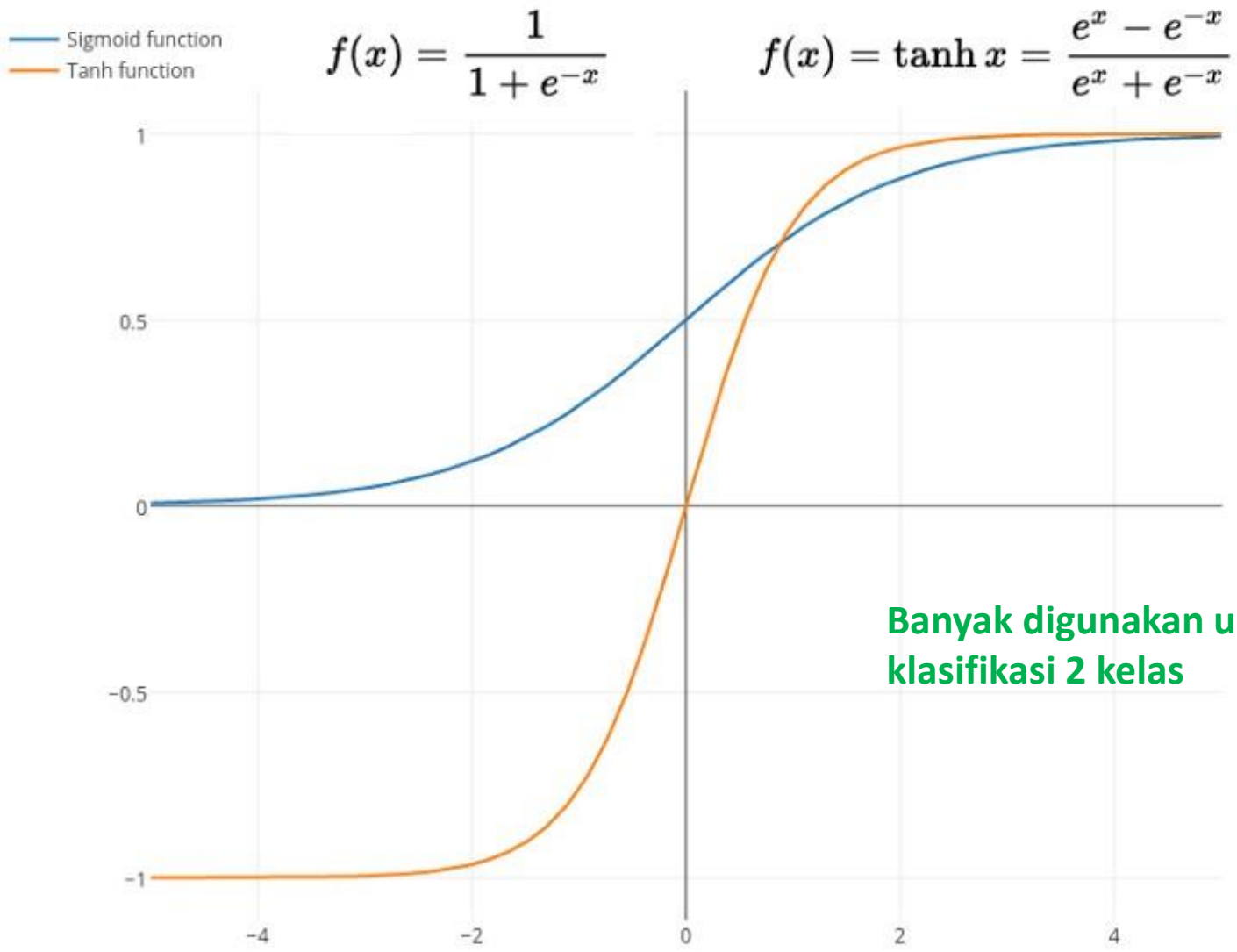
- Fungsi bipolar



- Fungsi bipolar dengan threshold



Fungsi Aktivasi Non Linear



Banyak digunakan untuk klasifikasi 2 kelas

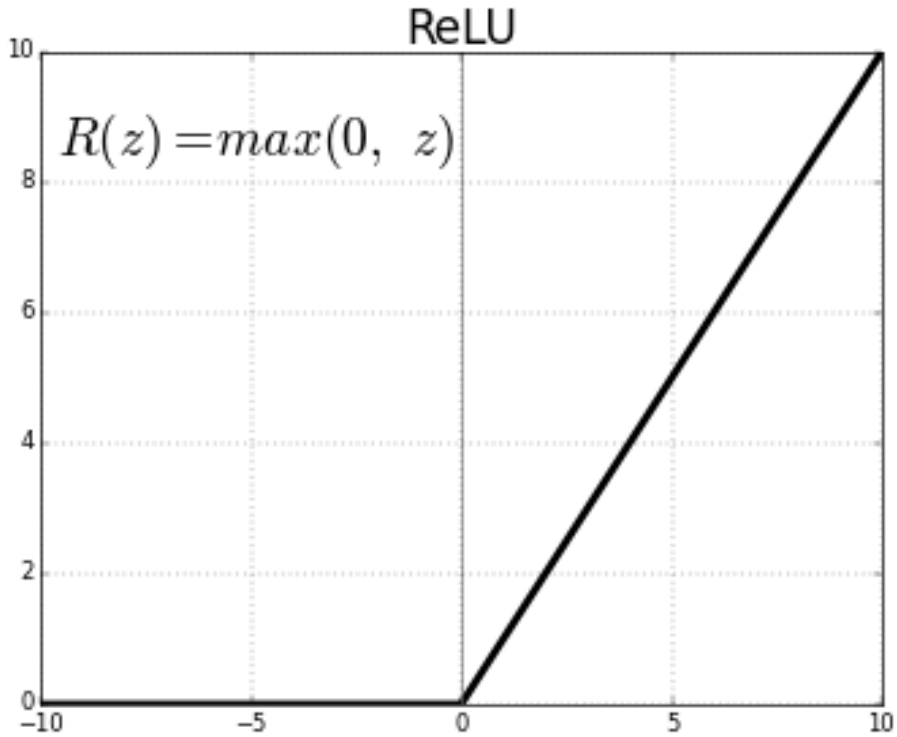
Fungsi Aktivasi Softmax

- Sama seperti sigmoid, namun total output adalah 1

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$

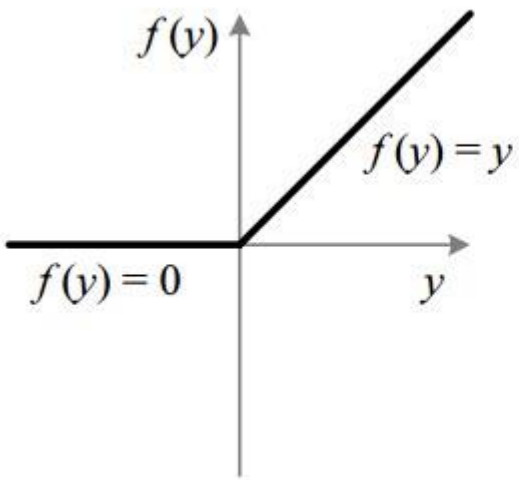
Fungsi Aktivasi Non Linear

Rectified Linear Unit

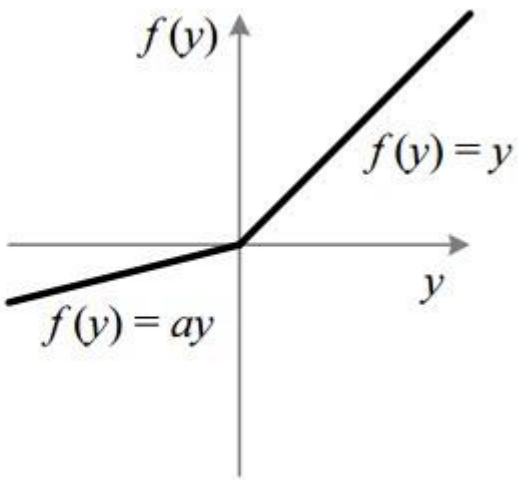


Fungsi Aktivasi Non Linear

ReLu



Leaky ReLu



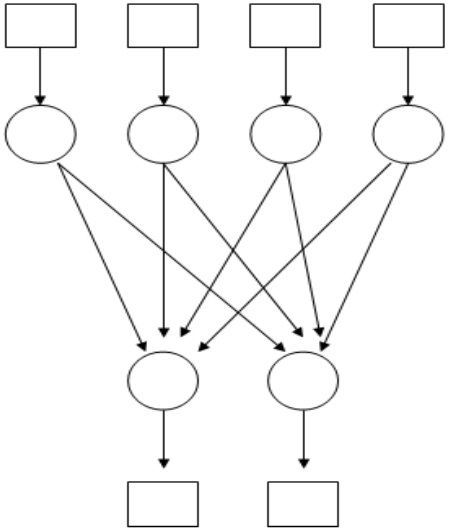
Nilai a biasanya 0.01 → Randomize ReLU

Arsitektur ANN

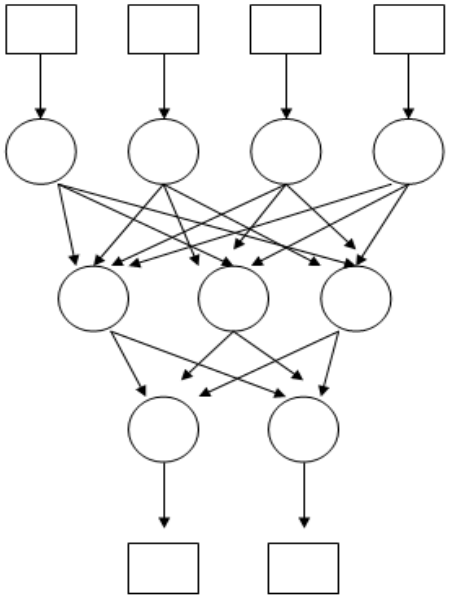
- Single Layer
  - Hanya memiliki satu lapisan dengan bobot-bobot terhubung.
  - Langsung menerima input dan mengolahnya menjadi output tanpa menggunakan hidden layer
- Multi Layer
  - Memiliki satu atau lebih lapisan input, satu atau lebih lapisan output, dan lapisan tersembunyi
  - Dapat menyelesaikan masalah yang lebih kompleks karena lebih akurat
  - Fungsi pembelajarannya lebih rumit
- Kompetitive Model / Recurrent Model
  - Hubungan antar neuron tidak diperlihatkan secara langsung pada arsitektur
  - Hubungan antar neuron dapat digambarkan sebagai jaring yang rumit

Arsitektur ANN

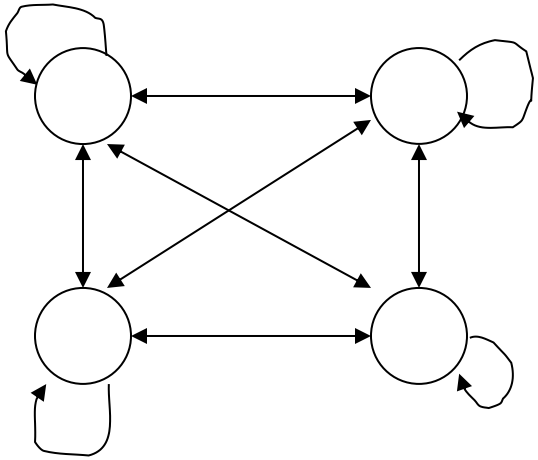
Single Layer



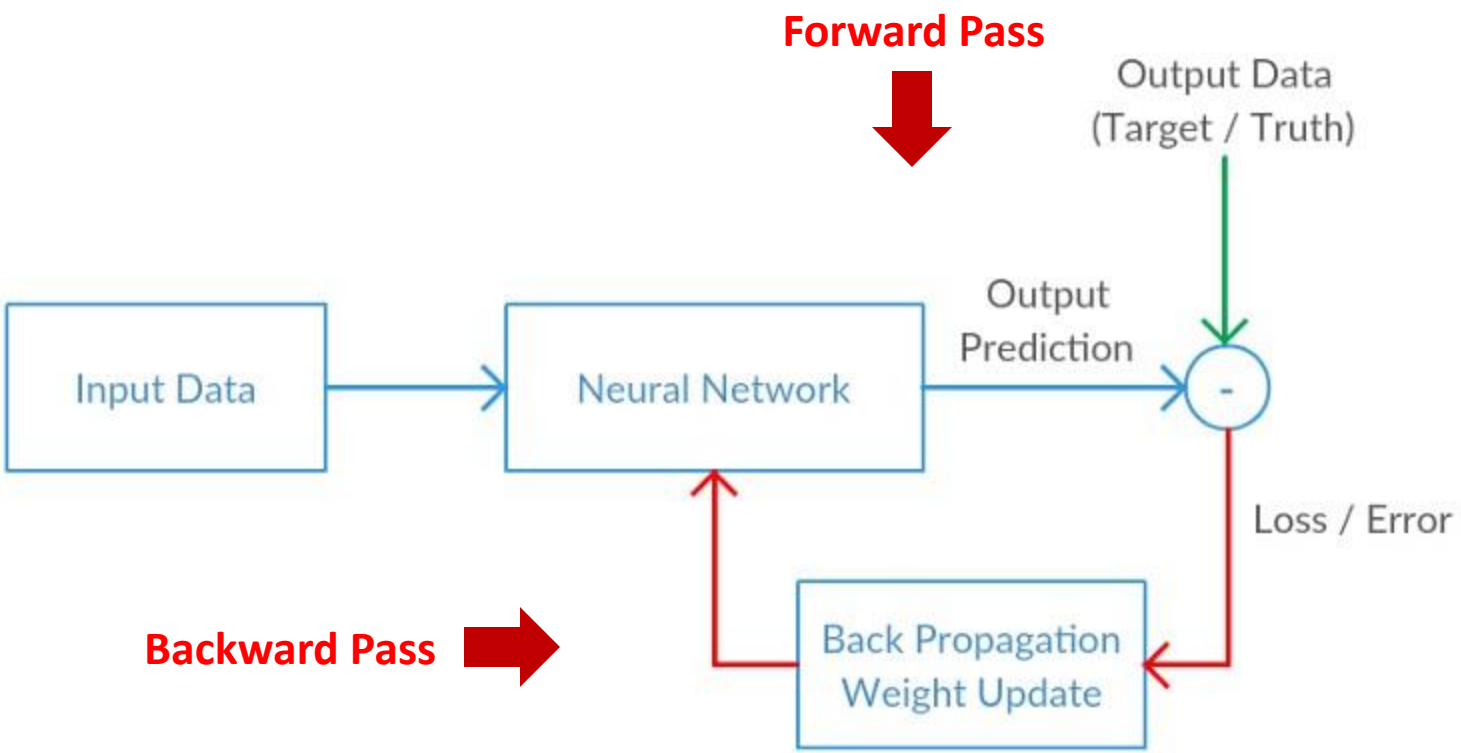
Multi Layer



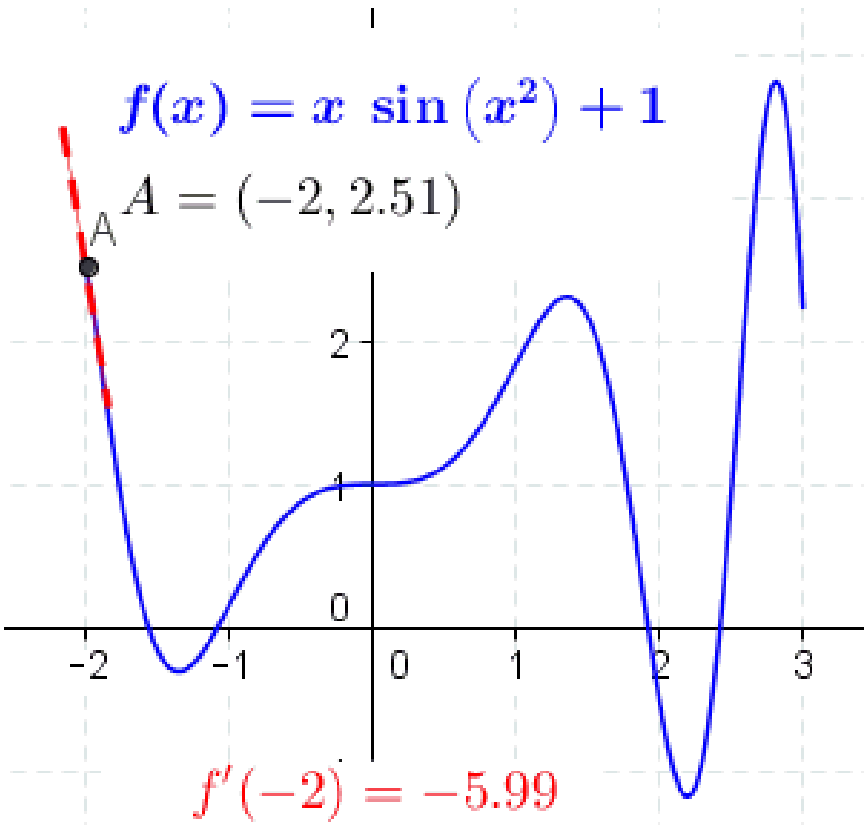
Competitive Layer/ Recurrent



Alur Proses Training



Backprop dengan Grad Descent



Prosedur Pemelajaran Umum

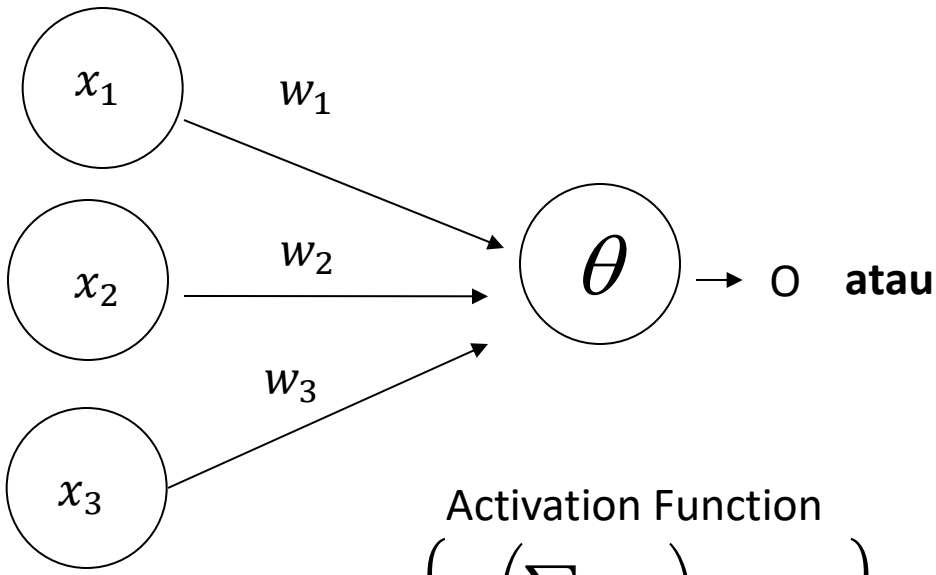
- Dimasukkan n data pelatihan
- Inisialisasi bobot-bobot jaringan, set i = 1
- Masukkan contoh ke-i ke dalam input
- Cari tingkat aktivasi unit output menggunakan algoritma yang ditetapkan
  - If memenuhi kriteria output then exit

    else:
- Update bobot-bobot menggunakan fungsi galat error, Bobot baru = bobot lama + delta
- If i=n then reset i=1, else i=i+1

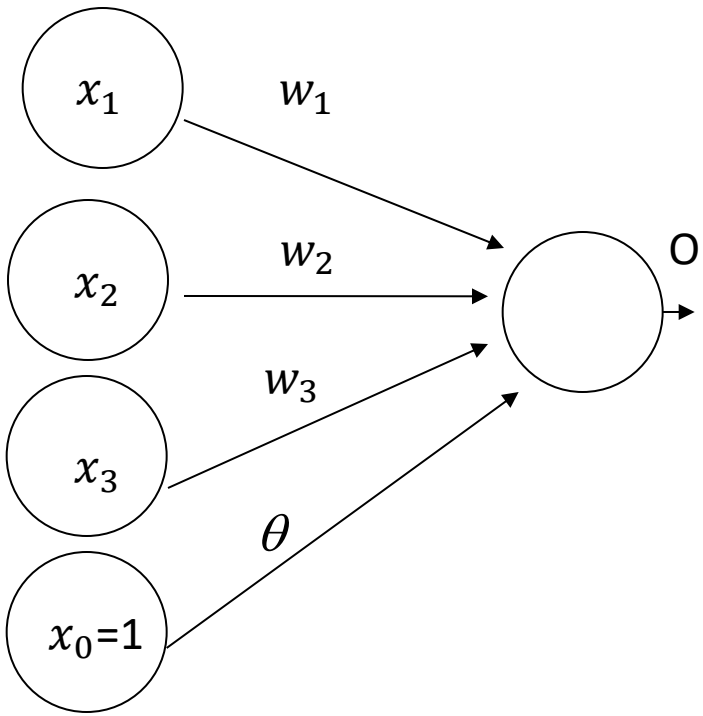
ANN: Perceptron

- Struktur awal usulan jaringan
- Rosenblatt, 50's and 60's

$\boldsymbol{\theta = bias = w_0}$



atau



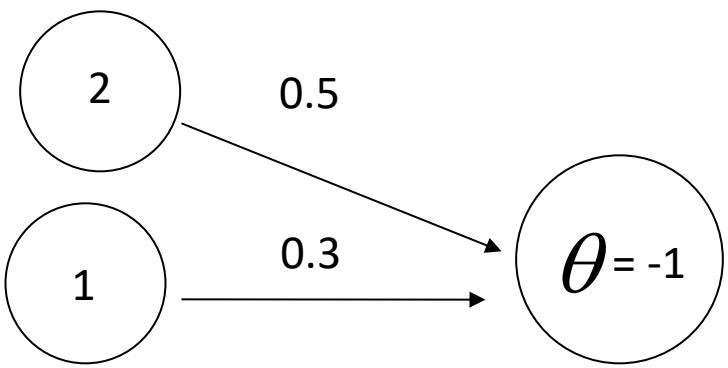
Activation Function

$$O = \begin{cases} 1: \left(\sum_i w_i x_i\right) + \theta > 0 \\ 0: otherwise \end{cases}$$

Perceptron

Algoritma Learning:

- Randomly assign weights (between 0-1)
- Present inputs from training data
- Get output O, nudge weights to gives results toward our desired output T
- Repeat; stop when no errors, or enough epochs completed



$$O = \left\{ \begin{array}{c} 1: \left(\sum_i w_i x_i \right) + \theta > 0 \\ 0: otherwise \end{array} \right\}$$

2(0.5) + 1(0.3) + (-1) = 0.3 → O=1

Perceptron Training

$$w_i(t+1) = w_i(t) + \Delta w_i(t)$$

$$\Delta w_i(t) = (T - O)\, x_i$$

Untuk semua bobot, termasuk Treshold. T=Label, O=Hasil Prediksi.

Example: T=0, O=1, W_1=0.5, W_2=0.3, x_1=2, x_2=1, W_θ=-1

$$w_1(t+1) = 0.5 + (0-1)(2) = -1.5$$
$$w_2(t+1) = 0.3 + (0-1)(1) = -0.7$$
$$w_\theta(t+1) = -1 + (0-1)(1) = -2$$

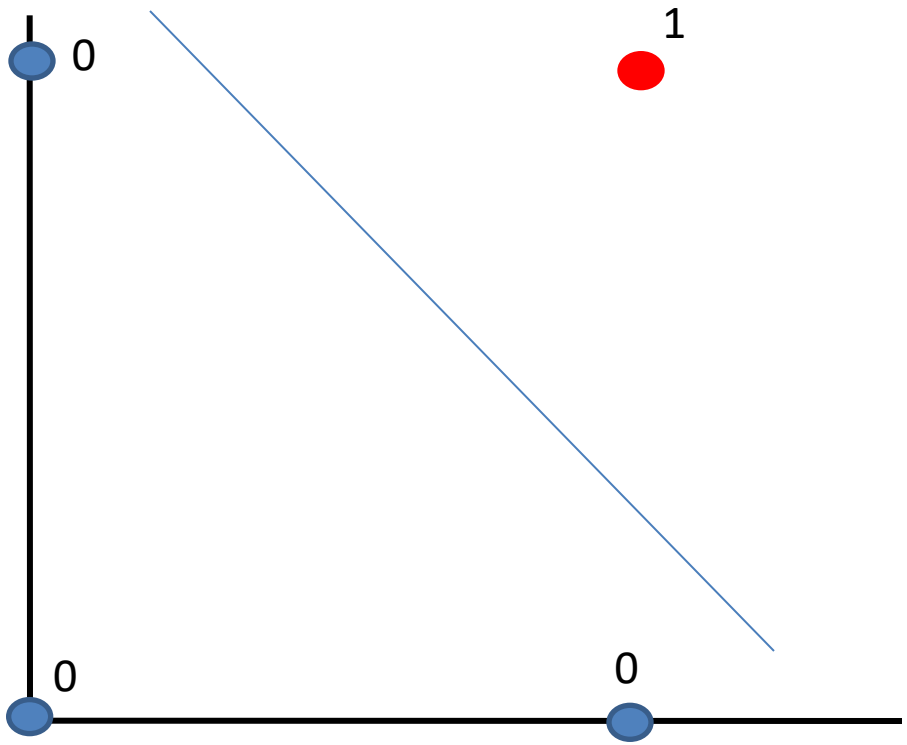
Jika kita hitung ulang dengan bobot baru, maka output sudah 0

Perceptron

- Pada dasarnya merupakan fungsi diskriminan linear
- Perceptron theorem: If a linear discriminant exists that can separate the classes without error, the training procedure is guaranteed to find that line or plane

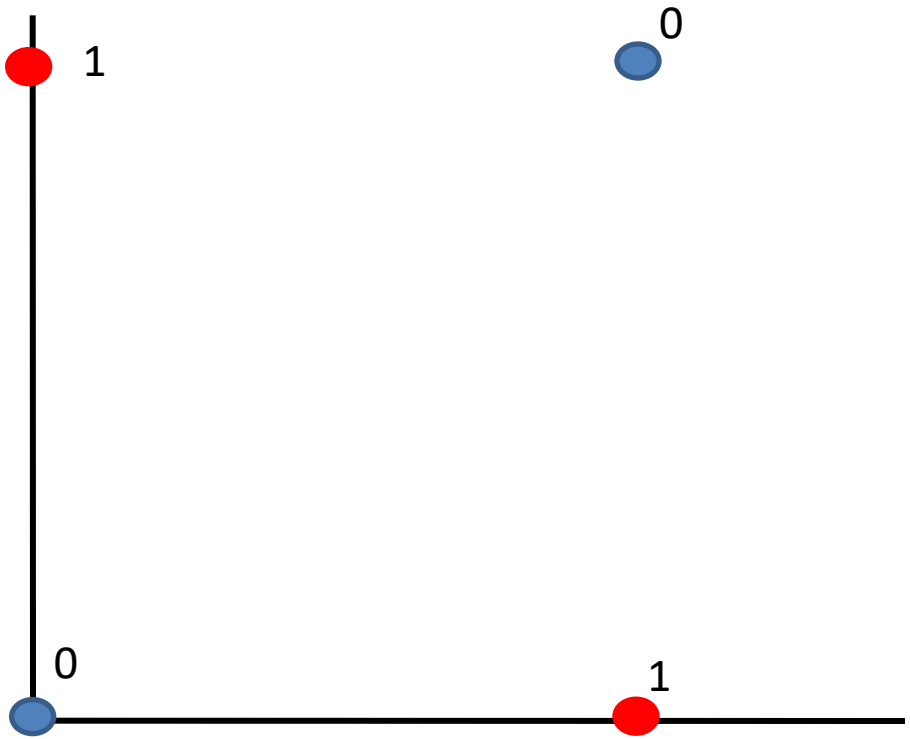
AND Problem

Input: 0,0 Output: 0
Input: 0,1 Output: 0
Input: 1,0 Output: 0
Input: 1,1 Output: 1



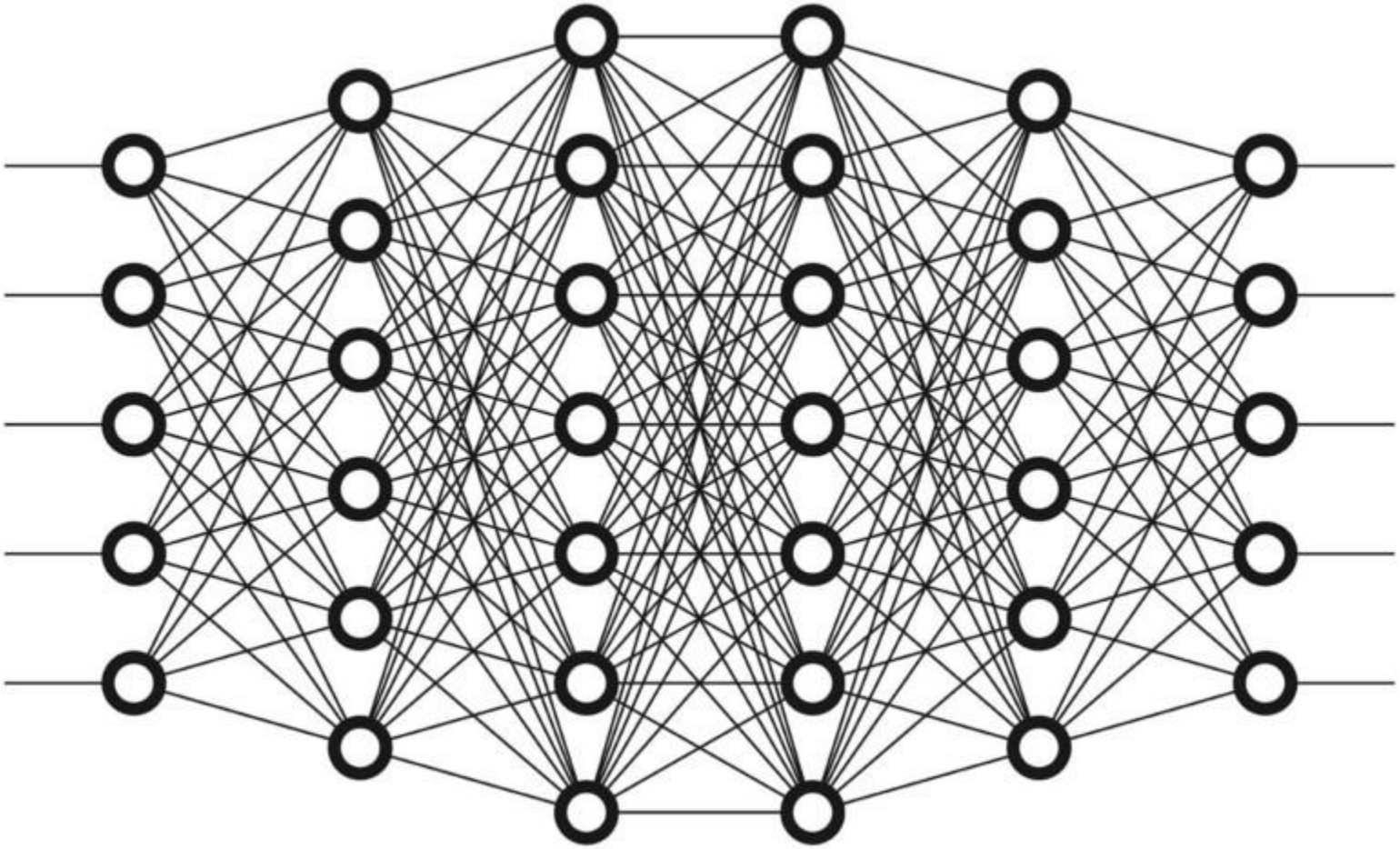
Exclusive Or (XOR) Problem

Input: 0,0 Output: 0
Input: 0,1 Output: 1
Input: 1,0 Output: 1
Input: 1,1 Output: 0

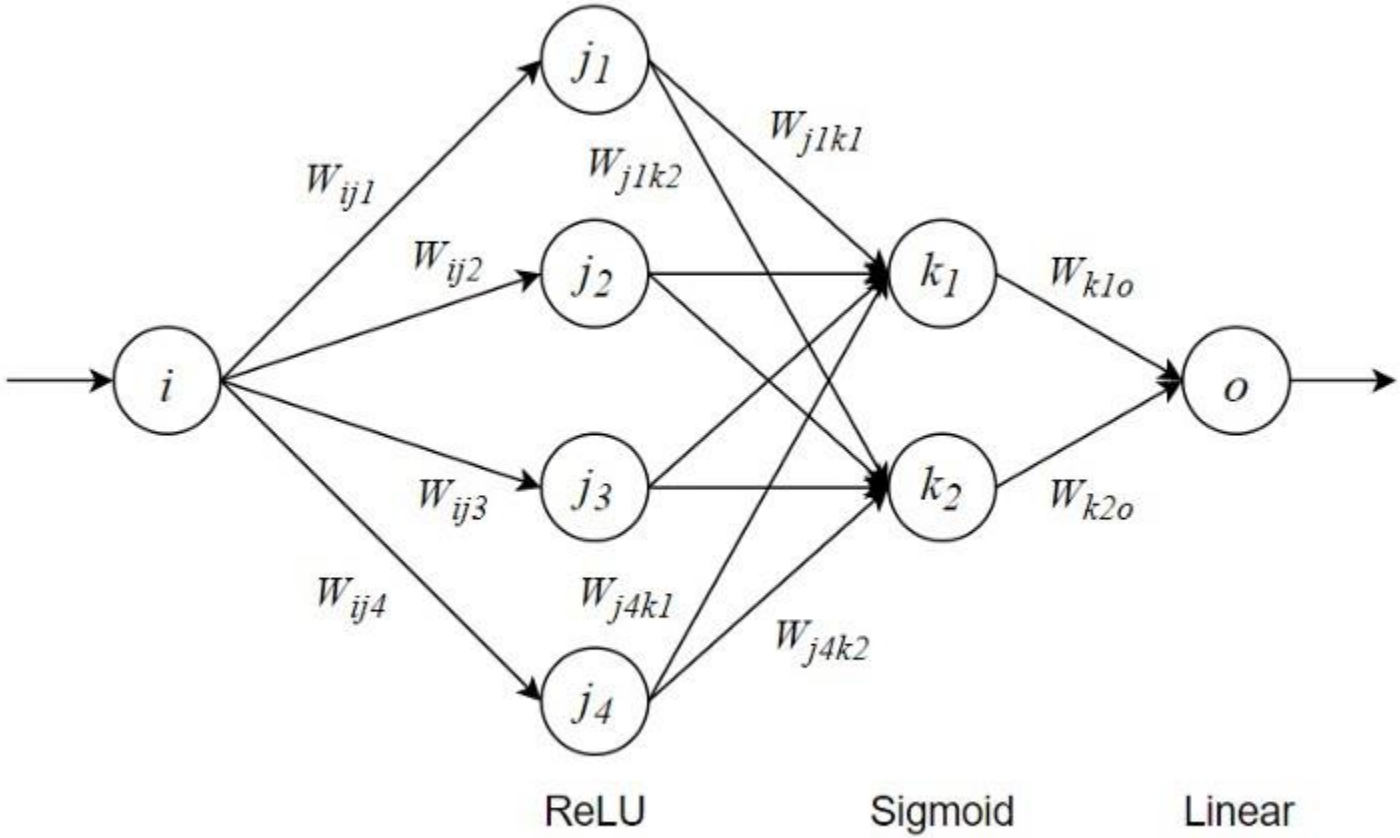


XOR Problem: Not Linearly Separable!
Tidak bisa diselesaikan dengan Single Layer Perceptron

Multi Layer Perceptron



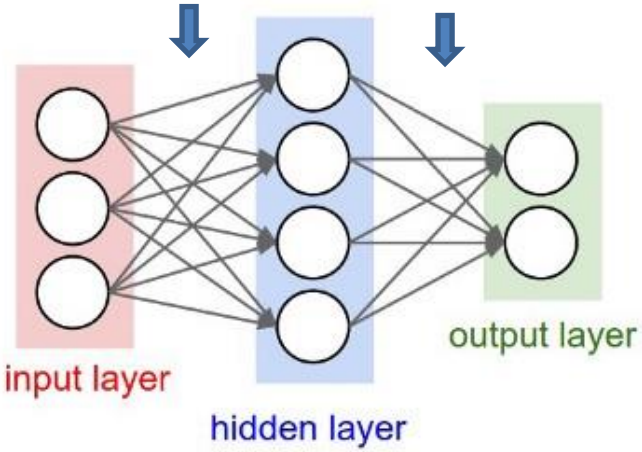
Contoh MLP


j_1
j_2
j_3
j_4
i
k_1
k_2
o
W_{ij1}
W_{ij2}
W_{ij3}
W_{ij4}
W_{j1k1}
W_{j1k2}
W_{j4k1}
W_{j4k2}
W_{k1o}
W_{k2o}
ReLU
Sigmoid
Linear

Multi Layer Perceptron Parameter

3x4 weight +4 bias

4x2 weight +2 bias

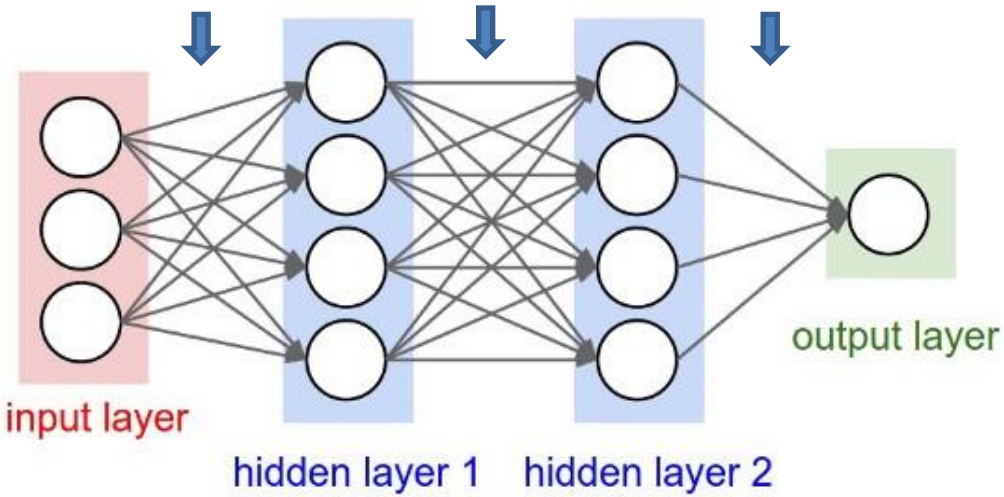


Jumlah Parameter:
12+4+8+2 = 26

3x4 weight +4 bias

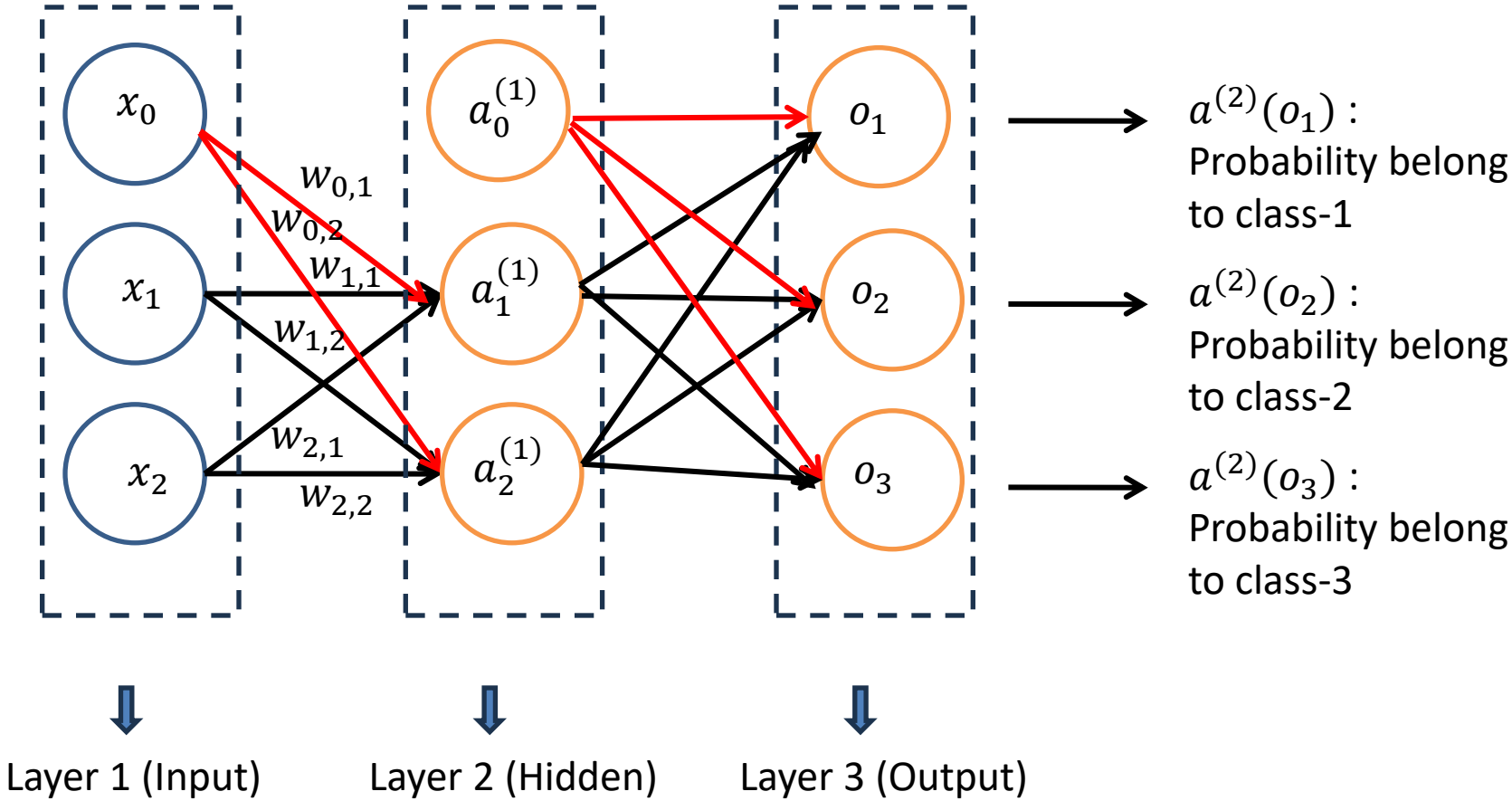
4x4 weight +4 bias

4x1 weight +1 bias

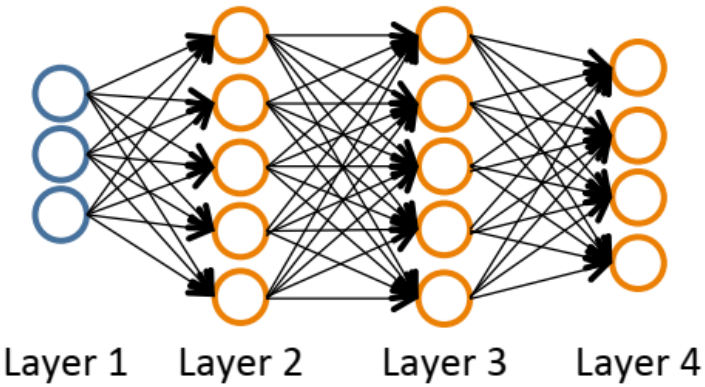


Jumlah Parameter:
12+4+16+4+4+1 = 41

Multiclass Classification



Binary vs Multiclass Classification



Dataset dengan m buah data:

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$$

L = jumlah layer dalam ANN

s_l = jumlah unit neuron pada layer ke-l

Binary Classification

$y = 0\ atau\ 1$

$Jml\ neuron\ output = 1$

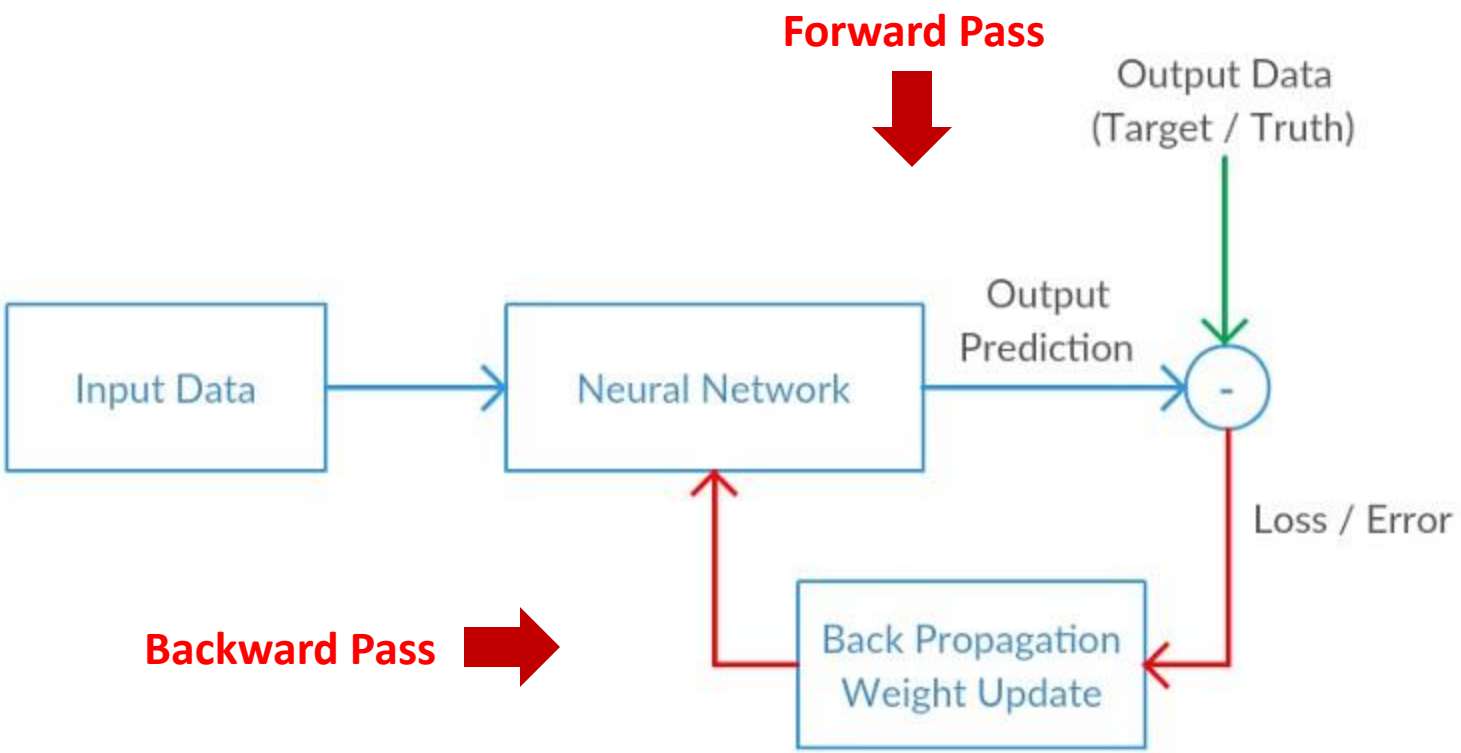
Multiclass Classification (untuk K buah kelas)

$y \in \mathbb{R}^K$ E.g. $\begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$

pedestrian car motorcycle truck

$Jml\ neuron\ output = jml\ kelas = K$

MLP: Alur Proses Training



Cost Function: Classification

- Logistic Regression

$$J(w) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log h_w\left(x^{(i)}\right) + \left(1 - y^{(i)}\right)\log(1 - h_w\left(x^{(i)}\right))\right]$$

- ANN/MLP

$$J(w) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log h_w\left(x^{(i)}\right)_k + \left(1 - y_k^{(i)}\right)\log(1 - h_w\left(x^{(i)}\right)_k)\right]$$

- Dimana:

$$h_w(x) = g(w^T x)$$

Gradient

Cost Function:

$$J(w) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log h_w\left(x^{(i)}\right)_k + \left(1 - y_k^{(i)}\right)\log(1 - h_w\left(x^{(i)}\right)_k)\right]$$

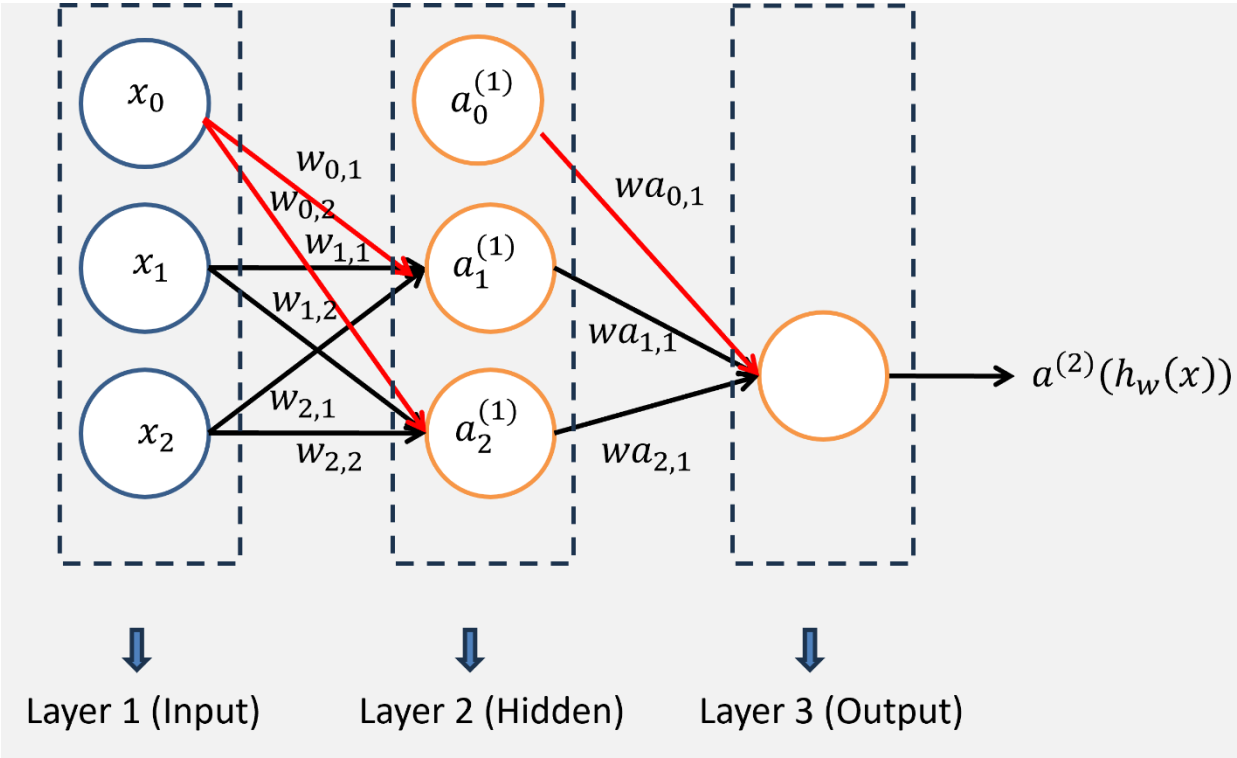
Goal: $\underset{w}{minimize} J(w)$

Gradient: $\frac{\partial J(w)}{\partial w_{i,j}^{l}} = ?$

Forward Pass

act_1 : *Activation Function* 1

act_2 : *Activation Function* 2



$$a_0^{(1)} = 1$$

$$a_1^{(1)} = act_1(w_{0,1}x_0 + w_{1,1}x_1 + w_{2,1}x_2) = act_1(W_1^{(1)}X)$$

$$a_2^{(1)} = act_1(w_{0,2}x_0 + w_{1,2}x_1 + w_{2,2}x_2) = act_1(W_2^{(1)}X)$$

$$\boldsymbol{a^{(1)} = act_1(W^{(1)}X)}$$

$$a^{(2)} = act_2(wa_{0,1}a_0^{(1)} + wa_{1,1}a_1^{(1)} + wa_{2,1}a_2^{(1)}) = act_2(W^{(2)}a^{(1)})$$

Forward Pass

$$a^{(1)} = x$$

$$z^{(2)} = W^{(1)} a^{(1)}$$

$$a^{(2)} = g\left(z^{(2)}\right) \qquad (add\ bias\ a_0^{(2)})$$

$$z^{(3)} = W^{(2)} a^{(2)}$$

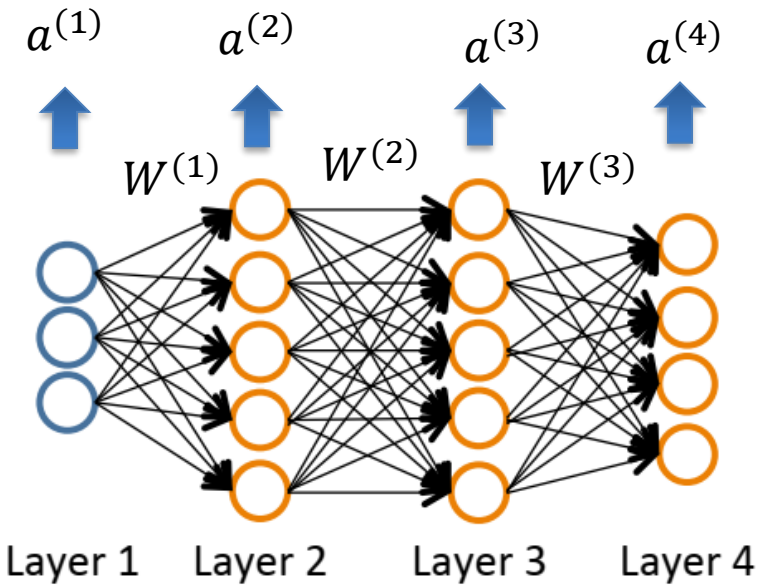
$$a^{(3)} = g\left(z^{(3)}\right) \qquad (add\ bias\ a_0^{(3)})$$

$$z^{(4)} = W^{(3)} a^{(3)}$$

$$a^{(4)} = h_w(x) = g\left(z^{(4)}\right)$$

Data:

(x, y)



Hitung Gradien: Backpropagation

$\delta_j^{(l)} = \text{`error` dari neuron ke-j pada layer ke-l}$

$$y = \begin{bmatrix} y_1 \\ \dots \\ y_j \end{bmatrix}$$

$Foreach\ output\ unit\ (L = 4)$

$$\delta_j^{(4)} = a_j^{(4)} - y_j$$

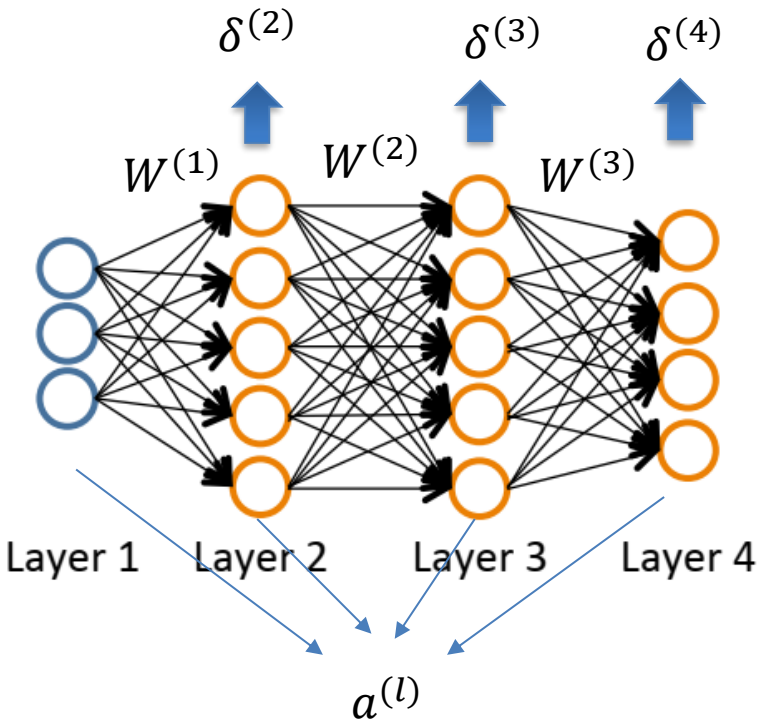
$$\delta^{(3)} = (W^{(3)})^T \delta^{(4)} .* g'\left(z^{(3)}\right)$$

$$\delta^{(2)} = (W^{(2)})^T \delta^{(3)} .* g'(z^{(2)})$$

$$a_j^{(4)} = (h_w\ (x))_j = g\left(z_j^{(4)}\right)$$

$$g'(z) = g(z).(1 - g(z))$$

$$\frac{\partial J(w)}{\partial w_{i,j}^l} = a_j^{(l)}.\delta_i^{(l+1)}$$



Backpropagation Algorithm

Training set $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$

Set $\triangle_{ij}^{(l)} = 0$ (for all l, i, j).

For $i = 1$ to m

Set $a^{(1)} = x^{(i)}$

Perform forward propagation to compute $a^{(l)}$ for $l = 2, 3, \ldots, L$

Using $y^{(i)}$, compute $\delta^{(L)} = a^{(L)} - y^{(i)}$

Compute $\delta^{(L-1)}, \delta^{(L-2)}, \ldots, \delta^{(2)}$

$\triangle_{ij}^{(l)} := \triangle_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$

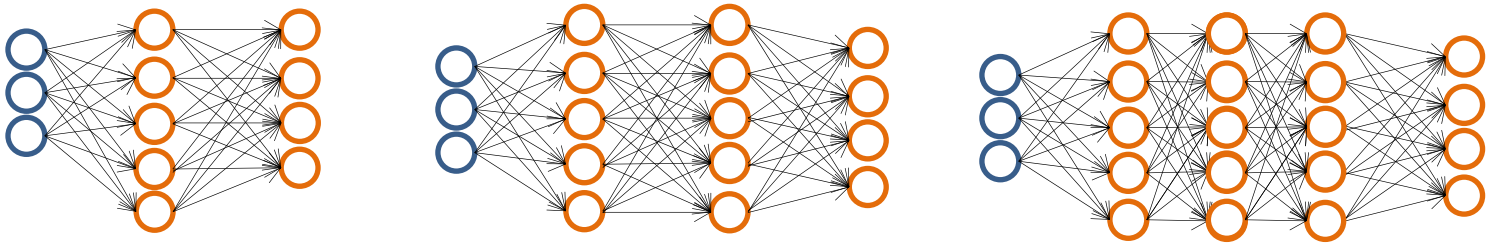
$D_{ij}^{(l)} := \frac{1}{m} \triangle_{ij}^{(l)} + \lambda \mathsf{W}_{ij}^{(l)}$ if $j \neq 0$

$D_{ij}^{(l)} := \frac{1}{m} \triangle_{ij}^{(l)}$ if $j = 0$

$$\frac{\partial}{\partial \mathsf{W}_{ij}^{(l)}} J(\mathsf{W}) = D_{ij}^{(l)}$$

ANN: Training

- Tentukan arsitektur yang digunakan

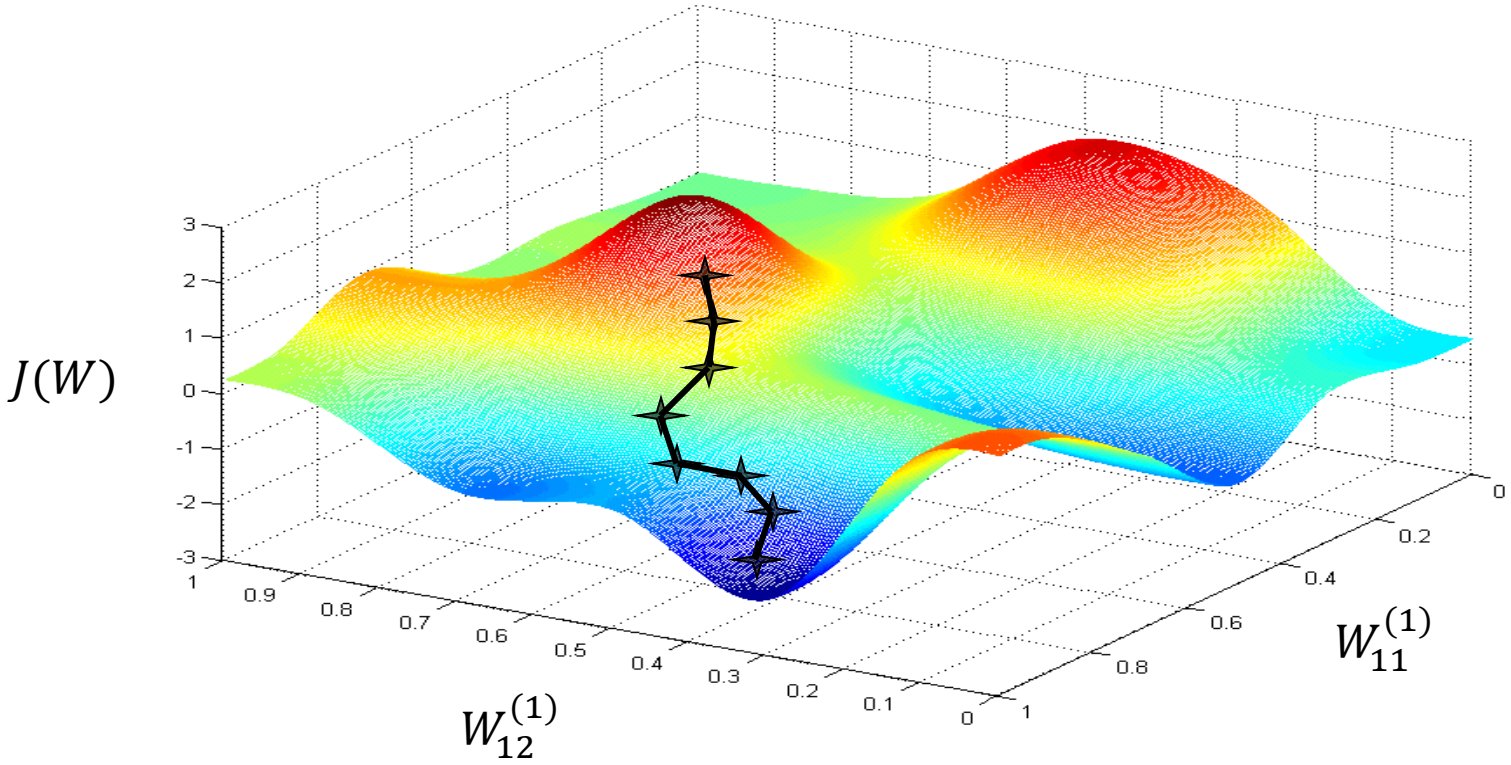


- Jml neuron pada layer input = dimensi fitur dari $x^{(i)}$
- Jml neuron pada layer output = Jml output yang diharapkan (mis. Jml Kelas dalam klasifikasi multiclass)
- Biasanya jumlah neuron pada setiap hidden layer, sama. Semakin banyak semakin bagus, semakin kompleks, semakin rawan overfitting.

ANN: Training

1. Inisialisasi bobot (W) secara random
2. Lakukan forward pass untuk dapat output $h_w\left(x^{(i)}\right)$ untuk sembarang data input $x^{(i)}$
3. Hitung cost $J(W)$
4. Lakukan backpropagation untuk menghitung partial derivatif $\frac{\partial J(w)}{\partial w_{i,j}^{l}}$
5. Update bobot dengan menggunakan gradient yang didapat, ulangi Langkah 1-5 sampai kriteria berhenti terpenuhi

ANN: Training

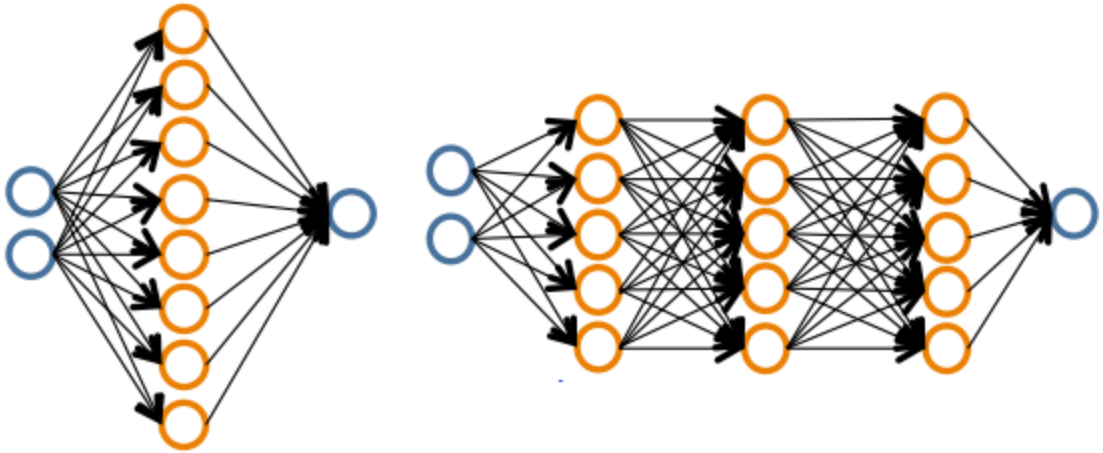


ANN and Overfitting

Small NN
(Parameter sedikit,
lebih rawan underfitting)

Komputasi Murah

Large NN
(Parameter banyak,
lebih rawan overfitting)



Komputasi Mahal

Gunakan parameter regularization
untuk menghindari overfitting

Thank You